CSL505 Cryptography — Term Paper
Indian Institute of Technology Bhilai, Winter 2026

MANTIS: Design, Analysis, and a Novel Variant of a Low-Latency Tweakable Block Cipher

Agastya Nath[1], Swarnadip Kar[2] and Divyansh Dubey[3]

[1] Department of Computer Science & Engineering,
Indian Institute of Technology Bhilai, Durg 491002, India.
agastyan@iitbhilai.ac.in
[2] Department of Computer Science & Engineering,
Indian Institute of Technology Bhilai, Durg 491002, India.
swarnadipk@iitbhilai.ac.in
[3] Department of Computer Science & Engineering,
Indian Institute of Technology Bhilai, Durg 491002, India.
divyanshd@iitbhilai.ac.in

Abstract. We present a study of **MANTIS**, a low-latency tweakable block cipher from the SKINNY family, proposed by Beierle et al. at CRYPTO 2016 [BJK+16]. MANTIS operates on 64-bit blocks with a 128-bit key and a 64-bit tweak, targeting memory encryption and other latency-sensitive applications. Its defining feature is the *α-reflection property*, which makes decryption essentially free given the encryption implementation. We present the complete specification—including the involutory MIDORI S-box [BBI+15], the cell permutation and MixColumns layer, the tweak schedule, and the FX-construction key schedule—along with a concrete encryption example verified against the official test vectors of [BJK+16]. Security analysis covers resistance to differential and linear attacks in both the single-key and related-tweak models, with lower bounds on active S-boxes reproduced from [BJK+16]. As a brownie-points contribution, we describe and analyse a *novel variant* of MANTIS that replaces the fixed round constants with an LFSR-based schedule seeded from the key and tweak, replaces the static k_1 round key with rotation-based sub-keys, and supports arbitrary half-round counts. Finally, we discuss how MANTIS and the wider SPN design philosophy apply to memory encryption, IoT authentication, and side-channel-resistant hardware.

Keywords: Block cipher · Tweakable block cipher · Lightweight cryptography · Low-latency · Memory encryption · α-reflection · SKINNY · MIDORI

1 Introduction

Block ciphers are the cornerstone of modern symmetric-key cryptography. The Advanced Encryption Standard (AES) [Nat01] is the de facto general-purpose block cipher, but its ten-round structure incurs substantial latency when implemented as a fully unrolled combinational circuit—a critical drawback for memory encryption, where every cache-line read or write must be protected within a single clock cycle [BCG+12, BJK+16]. Lightweight alternatives such as PRESENT [BKL+07] and SIMON [BSS+13] reduce gate count at the cost of throughput, and neither provides a *tweak* input suitable for sector-level data isolation.

The cipher **PRINCE** [BCG+12] was the first to address low-latency decryption explicitly by exploiting a symmetric "α-reflection" structure so that the decryption circuit

is almost identical to the encryption circuit. MANTIS, proposed by Beierle et al. at CRYPTO 2016 as part of the SKINNY cipher family [BJK+16], extends this idea by adding a 64-bit tweak input: it is a tweakable block cipher with PRINCE-comparable latency and area, built entirely from well-understood components of MIDORI [BBI+15].

The parameter r (half-round count) gives rise to the family MANTIS_5 through MANTIS_8, offering a spectrum of security-latency trade-offs. MANTIS_7 is the recommended production variant, claiming security equivalent to 2^{126-n} operations given 2^n chosen plaintext–ciphertext pairs [BJK+16].

Contributions.

1. Complete specification of MANTIS_r: key schedule, S-box, permutation and diffusion layers, tweak schedule, and encryption/decryption procedures.
2. A fully worked encryption example with hexadecimal traces, verified against the official test vectors from [BJK+16].
3. Differential and linear cryptanalysis of MANTIS, with active S-box bounds for MANTIS_5 through MANTIS_8 in both the single-key and related-tweak models.
4. A discussion of known structural weaknesses, including the related-key α-reflection distinguisher.
5. As a brownie-points contribution: a *novel variant* of MANTIS that uses an LFSR-seeded, key-and-tweak-dependent round-constant and sub-key schedule, supporting arbitrary half-round counts.
6. A discussion of real-world application domains for MANTIS and related designs.

Organisation. Section 2 establishes notation and background on tweakable block ciphers. Section 3 gives the complete MANTIS specification. Section 5 analyses its security. Section 6 presents the novel variant and slide-attack discussion. Section 7 discusses real-world relevance. Section 8 concludes.

2 Preliminaries

2.1 Notation

Throughout this paper we use the following conventions.

- $\mathbb{F}_2 = \{0, 1\}$ denotes the binary field.
- $\mathbb{F}_2^n$ denotes the n-dimensional vector space over $\mathbb{F}_2$ (i.e., bit strings of length n).
- $\oplus$ denotes bitwise XOR; $\|$ denotes concatenation.
- $[n] = \{0, 1, \ldots, n-1\}$.
- Hexadecimal literals are prefixed with `0x`; e.g., `0xA3`.
- A 64-bit value is viewed as 16 nibbles (4-bit cells), indexed from the most-significant nibble ($cell_0$) to $cell_{15}$, arranged in a 4×4 array row-wise.
- $\text{ROR}_{64}(x, n)$ denotes the 64-bit right rotation of x by n positions: $\text{ROR}_{64}(x, n) = (x \gg n) \oplus (x \ll (64 - n))$ truncated to 64 bits.
- $\text{MSB}(x)$ denotes the most-significant bit of x.

2.2 Tweakable Block Ciphers

Definition 1 (Tweakable Block Cipher [JNP14])**.** A *tweakable block cipher* (TBC) is a family of permutations $\widetilde{E} : \mathcal{K} \times \mathcal{T} \times \{0,1\}^n \to \{0,1\}^n$, where $\mathcal{K}$ is the key space, $\mathcal{T}$ is the *tweak* space, and n is the block size. For fixed $(K, T) \in \mathcal{K} \times \mathcal{T}$, the mapping $x \mapsto \widetilde{E}(K, T, x)$ is a permutation on $\{0,1\}^n$.

The principal advantage of a TBC over a standard block cipher is that the tweak can be changed freely without re-keying. This makes TBCs ideal for authenticated encryption, leakage-resilient designs, and memory encryption (where the memory address serves as the tweak) [BCG+12, BJK+16].

2.3 Substitution-Permutation Networks

A *Substitution-Permutation Network* (SPN) [Sti05] is an iterated block cipher that alternates two types of layers:

Definition 2 (Substitution Layer)**.** A *substitution layer* applies a bijective function $S : \mathbb{F}_2^m \to \mathbb{F}_2^m$, called an *S-box*, to each of ℓ non-overlapping m-bit chunks of the state. The block size is $n = m \cdot \ell$.

Definition 3 (Linear Layer)**.** A *linear layer* applies a linear map over $\mathbb{F}_2$ to the full state, ensuring that output bits of one S-box influence input bits of multiple S-boxes in the next round (*diffusion*).

MANTIS is an SPN cipher that reuses the MIDORI [BBI+15] round function components: the involutory S-box Sb_0, the cell permutation P, and the involutory MixColumns matrix M.

2.4 Security Notions

We consider resistance to the following classical structural attacks [BS91, Mat94]:

- **Differential cryptanalysis** [BS91]: exploits statistical biases in how plaintext differences propagate to ciphertext differences. Key metric: maximum differential probability $\mathrm{DP}^{\max}$ of the S-box.
- **Linear cryptanalysis** [Mat94]: exploits linear approximations between plaintext bits, ciphertext bits, and key bits. Key metric: maximum linear bias $\varepsilon^{\max}$ of the S-box.
- **Related-tweak attacks**: the attacker controls both the plaintext and the tweak, seeking to distinguish the TBC from a random permutation family.

2.5 The α-Reflection Property

The α-reflection property is inherited from PRINCE [BCG+12] and is central to MANTIS. Given the 192-bit extended key $(k_0 \| k_0' \| k_1)$, where $k_0' = \mathrm{ROR}_{64}(k_0, 1)$ and constant $\alpha =$ `0x243F6A8885A308D3`:

$$\mathrm{Dec}_{(k_0,\, k_0',\, k_1)}(\cdot, T) = \mathrm{Enc}_{(k_0',\, k_0,\, k_1 \oplus \alpha)}(\cdot, T). \tag{1}$$

This means the decryption circuit is essentially the same hardware as the encryption circuit with a slightly modified key [BJK+16], reducing decryption overhead to near zero.

3 The Cipher

3.1 Parameters

MANTIS_r is parameterised as shown in Table 1.

Table 1: MANTIS_r parameters [BJK+16].

Parameter	Value
Block size n	64 bits
Key size	128 bits ($k_0 \| k_1$, each 64 bits)
Tweak size	64 bits
Half-round count r	5 (MANTIS_5) to 8 (MANTIS_8)
Cell size	4 bits (nibble)
State layout	4×4 array of nibbles (row-wise)
Decryption overhead	Zero (same circuit, swapped keys)

The 128-bit key $k = k_0 \| k_1$ is extended to a 192-bit triple (k_0, k_0', k_1) with derived quantities:

$$k_0' = \text{ROR}_{64}(k_0, 1), \qquad \bar{k}_1 = k_1 \oplus \alpha, \qquad \alpha = \texttt{0x243F6A8885A308D3}.$$

3.2 The S-Box: Involutory MIDORI Sb_0

MANTIS uses the involutory 4-bit MIDORI S-box Sb_0, given in Table 2. An *involutory* S-box satisfies $\text{Sb}_0 \circ \text{Sb}_0 = \text{id}$, so the same table serves for both encryption and decryption [BBI+15, BJK+16].

Table 2: Involutory MIDORI S-box Sb_0 used in MANTIS [BJK+16].

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$\text{Sb}_0(x)$	C	A	D	3	E	B	F	7	8	9	1	5	0	2	4	6

The S-box has maximum differential probability $\text{DP}^{\max}(\text{Sb}_0) = 2^{-2}$ and maximum linear bias $\varepsilon^{\max}(\text{Sb}_0) = 2^{-2}$, which are optimal for involutory 4-bit S-boxes designed for low circuit depth [BBI+15, BJK+16].

3.3 PermuteCells

The 16 nibbles of the 4×4 state array are permuted by the MIDORI cell permutation P [BJK+16, BBI+15]. The forward permutation and its inverse are given in Table 3.

Table 3: MANTIS cell permutation P and its inverse P^{-1} [BJK+16].

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$P(i)$	0	11	6	13	10	1	12	7	5	14	3	8	15	4	9	2
$P^{-1}(i)$	0	5	15	10	13	8	2	7	11	14	4	1	6	3	9	12

The MIDORI permutation was chosen to maximise active S-box counts compared to the PRINCE permutation, while keeping a branch-number-4 diffusion matrix [BJK+16].

3.4 MixColumns

Each column of the 4×4 state array is multiplied by the following binary matrix M over GF(2) [BJK+16, BBI+15]:

$$M = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

Since $M^2 = I$ over GF(2), M is *involutory*: the same circuit is used in both forward rounds and inverse rounds. The state layout is cell $s[4r + c]$ at row r, column c for $r, c \in [4]$.

3.5 Tweak Schedule

The 64-bit tweak T (16 nibbles) is updated each round by the permutation h defined as [BJK+16]:

$$h = [6,\ 5,\ 14,\ 15,\ 0,\ 1,\ 2,\ 3,\ 7,\ 12,\ 13,\ 4,\ 8,\ 9,\ 10,\ 11].$$

The action is $h(T)[i] = T[h[i]]$. The precomputed sequence $h^1(T), h^2(T), \ldots, h^r(T)$ is used in rounds $1, 2, \ldots, r$. Crucially, the permutation h itself requires no additional Galois-field multiplications, keeping the tweak schedule lightweight [BJK+16].

3.6 Round Constants

Eight 64-bit round constants $\mathrm{RC}_1, \ldots, \mathrm{RC}_8$ are fixed and derived from the hexadecimal expansion of π [BJK+16]. They are listed in Table 4.

Table 4: MANTIS round constants RC_1–RC_8 [BJK+16].

Round i	Round constant RC_i (hexadecimal)
1	`0x13198A2E03707344`
2	`0xA4093822299F31D0`
3	`0x082EFA98EC4E6C89`
4	`0x452821E638D01377`
5	`0xBE5466CF34E90C6C`
6	`0xC0AC29B7C97C50DD`
7	`0x3F84D5B5B5470917`
8	`0x9216D5D98979FB1B`

The constant $\alpha =$ `0x243F6A8885A308D3` (also from π) is XORed with k_1 to form $\bar{k}_1 = k_1 \oplus \alpha$, used in the backward half.

3.7 Round Function

The forward round $R_i(\cdot, tk)$ for tweakey tk is [BJK+16]:

$$R_i = \mathrm{MixColumns} \circ \mathrm{PermuteCells} \circ \mathrm{AddTweakey}_{tk} \circ \mathrm{AddConstant}_i \circ \mathrm{SubCells}. \quad (2)$$

The inverse round R_i^{-1} reverses the order, using $M^{-1} = M$ (involutory), P^{-1}, and $\mathrm{Sb}_0^{-1} = \mathrm{Sb}_0$:

$$R_i^{-1} = \mathrm{SubCells} \circ \mathrm{AddConstant}_i \circ \mathrm{AddTweakey}_{tk} \circ \mathrm{PermuteCells}^{-1} \circ \mathrm{MixColumns}. \quad (3)$$

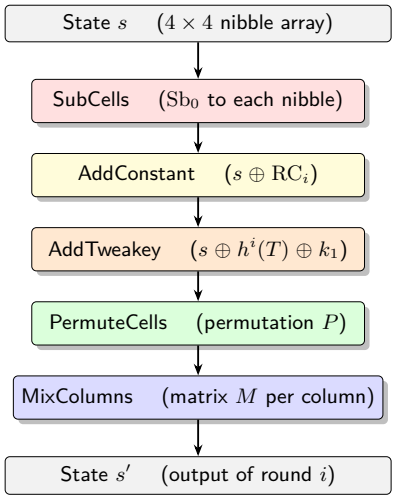

Figure 1: MANTIS forward round R_i: SubCells $\to$ AddConstant $\to$ AddTweakey $\to$ PermuteCells $\to$ MixColumns [BJK+16]. All five components are invertible with the same or minimal extra hardware.

3.8 Overall Encryption Structure and Key Schedule

MANTIS uses an FX-construction [BCG+12, BJK+16] with pre- and post-whitening keys. Define:

$$H_r(\cdot, T, k_1) = \mathrm{SubCells} \circ R_r(\cdot, h^r(T) \oplus k_1) \circ \cdots \circ R_1(\cdot, h^1(T) \oplus k_1),$$

$$H_r^{-1}(\cdot, T, \bar{k}_1) = R_1^{-1}(\cdot, h^1(T) \oplus \bar{k}_1) \circ \cdots \circ R_r^{-1}(\cdot, h^r(T) \oplus \bar{k}_1) \circ \mathrm{SubCells}.$$

Then the full encryption is [BJK+16]:

$$c = \big(H_r^{-1}(M(H_r(m \oplus k_0 \oplus k_1 \oplus T)))\big) \oplus k_0' \oplus k_1 \oplus \alpha \oplus T. \tag{4}$$

Algorithm 1 presents this as pseudocode.

3.9 Decryption via the α-Reflection Property

Decryption is performed by invoking the encryption routine with a swapped key triple. By (1) [BJK+16]:

$$\mathrm{Dec}_{(k_0,\, k_0',\, k_1)}(c,\, T) = \mathrm{Enc}_{(k_0',\, k_0,\, k_1 \oplus \alpha)}(c,\, T).$$

No separate decryption circuit is required; only the key inputs are swapped. This is the key advantage of the reflection design paradigm [BCG+12].

3.10 Worked Example

We verify MANTIS_5 using the official test vectors from Appendix B.2 of [BJK+16].

Inputs.

$$k_0 = \texttt{0x92F09952C625E3E9},$$
$$k_1 = \texttt{0xD7A060F714C0292B},$$
$$T = \texttt{0xBA912E6F1055FED2}.$$

Algorithm 1 MANTIS$_r$ Encryption [BJK+16]

Require: plaintext $m \in \{0,1\}^{64}$; key $k_0 \| k_1$ (128 bits); tweak T (64 bits); half-round count $r \in \{5,6,7,8\}$
Ensure: ciphertext $c \in \{0,1\}^{64}$

```
k'_0 ← ROR_64(k_0, 1);   k̄_1 ← k_1 ⊕ α
Compute T^(i) ← h^i(T) for i = 1, ..., r
s ← m ⊕ k_0 ⊕ k_1 ⊕ T                          ▷ Initial whitening
for i = 1 to r do                              ▷ Forward half H_r
    tk ← T^(i) ⊕ k_1
    s ← SubCells(s)
    s ← s ⊕ RC_i                               ▷ AddConstant
    s ← s ⊕ tk                                 ▷ AddTweakey
    s ← P(s)                                   ▷ PermuteCells
    s ← M(s)                                   ▷ MixColumns
end for
s ← SubCells(s)                                ▷ Closing SubCells of H_r
s ← M(s)                                       ▷ Middle MixColumns layer
s ← SubCells(s)                                ▷ Opening SubCells of H_r^-1
for i = r downto 1 do                          ▷ Backward half H_r^-1
    tk ← T^(i) ⊕ k̄_1
    s ← M(s)                                   ▷ MixColumns (M^-1 = M)
    s ← P^-1(s)                                ▷ PermuteCells^-1
    s ← s ⊕ tk                                 ▷ AddTweakey (self-inverse)
    s ← s ⊕ RC_i                               ▷ AddConstant (self-inverse)
    s ← SubCells(s)                            ▷ SubCells (Sb_0^-1 = Sb_0)
end for
c ← s ⊕ k'_0 ⊕ k_1 ⊕ α ⊕ T                     ▷ Final whitening return c
```

Derived keys.

$$k'_0 = \mathrm{ROR}_{64}(k_0, 1) = \texttt{0xC97804A9631AF1F4},$$
$$\bar{k}_1 = k_1 \oplus \alpha = \texttt{0xD7A060F714C0292B} \oplus \texttt{0x243F6A8885A308D3} = \texttt{0xF39F0A7F916321F8}.$$

MANTIS$_5$ result.

$$m = \texttt{0x3B5C77A4921F9718},$$
$$c = \mathrm{Enc}_{(k_0,k'_0,k_1)}(m, T) = \texttt{0xD6522035C1C0C6C1}.$$

This matches Appendix B.2 of [BJK+16]. ✓

Decryption check. Invoking $\mathrm{Enc}_{(k'_0,k_0,\bar{k}_1)}(\texttt{0xD6522035C1C0C6C1}, T)$ returns `0x3B5C77A4921F9718`. ✓

Additional official test vectors for MANTIS$_6$–MANTIS$_8$ are given in Table 5.

4 Proposed Novel Cipher

The original MANTIS block cipher is designed as a lightweight tweakable block cipher optimized for low-latency applications. Although the cipher achieves efficient hardware-oriented encryption through its reflection-based structure, it employs fixed round constants and a static round-key usage pattern throughout encryption and decryption rounds. Such static structures may introduce predictable differential propagation behavior across rounds.

Table 5: MANTIS test vectors from [BJK+16] (Key = `92F09952C625E3E9 D7A060F714C0292B`, Tweak = `BA912E6F1055FED2`).

Variant	Plaintext (hex)	Ciphertext (hex)
MANTIS_5	`3B5C77A4921F9718`	`D6522035C1C0C6C1`
MANTIS_6	`D6522035C1C0C6C1`	`60E43457311936FD`
MANTIS_7	`60E43457311936FD`	`308E8A07F168F517`
MANTIS_8	`308E8A07F168F517`	`971EA01A86B410BB`

To improve round diversity and enhance diffusion characteristics while preserving the lightweight architecture of MANTIS, a modified variant of the cipher is proposed. The proposed design introduces two primary modifications:

- Dynamic round constant generation using a 64-bit Linear Feedback Shift Register (LFSR).
- A rotating round-key schedule derived from the master key component K_1.

The overall substitution-permutation network structure, tweak permutation, and reflection architecture of MANTIS are preserved in the proposed design.

4.1 Dynamic Round Constant Generation

In the original MANTIS cipher, fixed round constants derived from hexadecimal digits of π are used during round transformations. In the proposed modification, these constants are replaced by dynamically generated round constants using a 64-bit LFSR.

The feedback polynomial used for the LFSR is:

$$p(x) = x^{64} + x^4 + x^3 + x + 1$$

The LFSR evolves according to:

$$RC_{i+1} = LFSR(RC_i)$$

where RC_i denotes the round constant for the ith round.

The chosen feedback polynomial is primitive over $GF(2)$, yielding a maximal theoretical period of:

$$2^{64} - 1$$

for all nonzero initialization states.

The initial seed for round constant generation is derived using:

$$RC_0 = K_0 \oplus T$$

where K_0 represents the upper 64-bit master key segment and T denotes the tweak value.

The use of dynamic round constants increases round asymmetry and reduces repetitive differential propagation structures while maintaining lightweight implementation complexity.

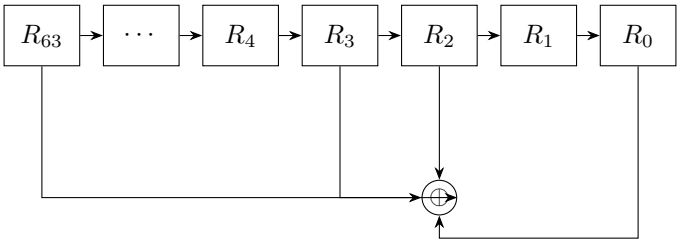

Figure 2: 64-bit LFSR used for dynamic round constant generation.

4.2 Rotating Round-Key Schedule

The original MANTIS cipher uses a static round-key structure derived from the lower master-key half K_1. In the proposed modification, dynamic round keys are generated through nibble-wise cyclic rotations combined with the generated round constants.

The proposed round-key generation scheme is defined as:

$$\mathrm{RK}_i = \mathrm{ROR}_{64}(K_1, 4i) \oplus RC_i$$

where:

- ROR_{64} denotes 64-bit right rotation,
- i is the round index,
- RC_i is the generated round constant.

The corresponding round tweakey becomes:

$$TK_i = h_i(T) \oplus RK_i$$

Generation of round-key and round tweakey is shown graphically in 3.

The backward reflected phase preserves the original α-reflection structure of MANTIS by employing:

$$RK_i \oplus \alpha$$

during inverse round processing.

The proposed schedule preserves the lightweight design philosophy while introducing increased round diversity and reducing static key reuse across rounds.

Table 6: Generated 64-bit round-keys used in the proposed design for 8 forward rounds.

Round Number	Round-key
1	`951BB132A73C1FA9`
2	`7B14CE1B5BF4FA5F`
3	`333BA6F156B138EF`
4	`6A266E4ED377FD1A`
5	`8489CEA7610EA2F9`
6	`CC1FCC6D6E63507F`
7	`546F5DC8E60141A7`
8	`241BB7C0EFAEFD5A`

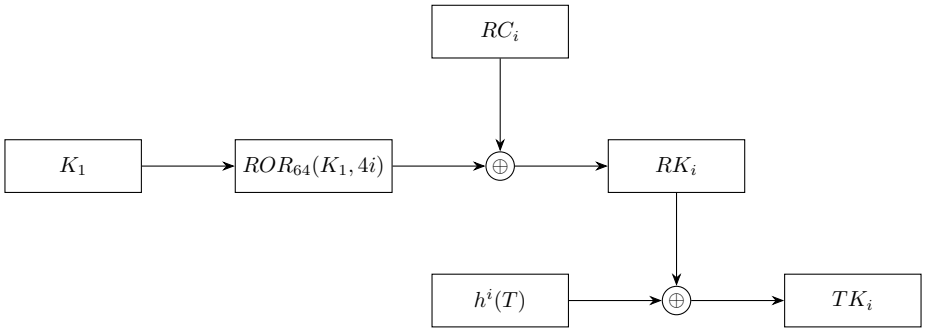

Figure 3: Dynamic round-key and tweakey generation process for the proposed modified MANTIS cipher.

4.3 Performance Evaluation

Performance benchmarking was conducted to evaluate the computational overhead introduced by the proposed modifications to the original MANTIS cipher. The experiments were performed under identical software conditions using repeated block encryption operations for both cipher variants.

The original MANTIS implementation achieved a total execution time of approximately 9.17 seconds for the performed benchmark iterations, corresponding to an average encryption time of approximately 91.65 microseconds per block and a throughput of approximately 10,910 encryptions per second.

The modified MANTIS implementation achieved a total execution time of approximately 11.13 seconds, corresponding to an average encryption time of approximately 111.26 microseconds per block and a throughput of approximately 8,988 encryptions per second (included in 7).

The obtained results indicate that the proposed modifications introduce a relative computational overhead of approximately 21.39% compared to the original MANTIS implementation.

The observed increase in execution time is primarily attributed to the additional operations required for dynamic round-constant generation using the LFSR mechanism and the rotating round-key schedule computation performed during encryption rounds. Despite this additional overhead, the modified cipher maintains sufficiently high throughput and preserves the lightweight operational characteristics of the original MANTIS architecture.

The performance results demonstrate that the proposed modifications improve diffusion-related security indicators while introducing only moderate computational cost. This tradeoff is considered acceptable within the context of lightweight cryptographic experimentation, particularly given the observed improvements in active S-box participation and differential propagation behavior.

It should also be noted that the presented benchmark values were obtained using a Python software implementation. Since Python introduces substantial interpreter overhead for bitwise and low-level arithmetic operations, the measured timing results are not representative of optimized implementations in lower-level environments such as C, FPGA, or ASIC-based realizations.

Table 7: Performance comparison between the original and modified MANTIS implementations.

Metric	Original MANTIS	Modified MANTIS
Total Execution Time	9.165 s	11.126 s
Average Encryption Time	91.65 μs/block	111.26 μs/block
Throughput	10,910 encryptions/s	8,988 encryptions/s
Relative Overhead	—	21.39%

5 Security Analysis

5.1 Differential Cryptanalysis

Differential cryptanalysis was performed to evaluate the diffusion characteristics and differential propagation behavior of the proposed modified MANTIS cipher. Since the substitution layer of the cipher remains unchanged, the local differential properties remain identical to the original MANTIS design. However, the proposed modifications affect global propagation behavior across rounds.

For subsequent analysis in this section, the following values will be considered:

- K = `0x92F09952C625E3E9D7A060F714C0292B`
- Tweak = `0xBA912E6F1055FED2`
- Rounds = 7 (`MANTIS_7`)

5.1.1 S-box Differential Properties

The proposed cipher retains the involutory 4-bit MIDORI Sb_0 substitution box used in the original MANTIS cipher. To evaluate the local differential characteristics of the S-box, a Differential Distribution Table (DDT) was generated experimentally.

The DDT entry is defined as:

$$\mathrm{DDT}[\Delta x][\Delta y]$$

which represents the number of input values satisfying:

$$S(x) \oplus S(x \oplus \Delta x) = \Delta y$$

The experimentally generated DDT showed a maximum nontrivial entry value of:

$$4$$

resulting in a differential uniformity of:

$$4$$

and a maximum differential probability of:

$$2^{-2}$$

Since the proposed modifications do not alter the substitution layer, both the original and modified MANTIS ciphers exhibit identical local differential properties.

S-box differential uniformity. For the involutory MIDORI S-box Sb_0, every non-trivial entry of the Difference Distribution Table (DDT) is at most 4 [BJK+16, BBI+15]. Hence:

$$\mathrm{DP}^{\max}(\mathrm{Sb}_0) = \frac{4}{2^4} = 2^{-2}.$$

Table 8: Differential Distribution Matrix

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	2	4	0	2	2	2	0	2	0	0	0	0	0	2	0
2	0	4	0	0	4	0	0	0	0	4	0	0	4	0	0	0
3	0	0	0	0	2	0	4	2	2	2	0	0	0	2	0	2
4	0	2	4	2	2	2	0	0	2	0	0	2	0	0	0	0
5	0	2	0	0	2	0	0	4	0	2	4	0	2	0	0	0
6	0	2	0	4	0	0	0	2	2	0	0	0	2	2	0	2
7	0	0	0	2	0	4	2	0	0	0	0	2	0	4	2	0
8	0	2	0	2	2	0	2	0	0	2	0	2	2	0	2	0
9	0	0	4	2	0	2	0	0	2	2	0	2	2	0	0	0
10	0	0	0	0	0	4	0	0	0	0	4	0	0	4	0	4
11	0	0	0	0	2	0	0	2	2	2	0	4	0	2	0	2
12	0	0	4	0	0	2	2	0	2	2	0	0	2	0	2	0
13	0	0	0	2	0	0	2	4	0	0	4	2	0	0	2	0
14	0	2	0	0	0	0	0	2	2	0	0	0	2	2	4	2
15	0	0	0	2	0	0	2	0	0	0	4	2	0	0	2	4

5.1.2 Active S-box Analysis

Active S-box analysis was performed to evaluate differential propagation across encryption rounds. A one-bit plaintext differential was introduced at the cipher input, and the number of active S-boxes entering each substitution layer was measured throughout the encryption process.

An S-box was considered active when its input difference was nonzero.

The original MANTIS cipher produced a total active S-box count of:

$$204$$

whereas the proposed modified cipher produced:

$$213$$

active S-boxes under the same experimental conditions.

The modified cipher demonstrated stronger sustained activity in later forward and backward layers, indicating improved diffusion persistence and reduced differential cancellation effects (9).

5.1.3 Avalanche Analysis

Avalanche analysis was performed by flipping a single plaintext bit and measuring the number of changed ciphertext bits.

The avalanche metric is defined as:

$$HW(C_1 \oplus C_2)$$

where:

- HW denotes Hamming weight.

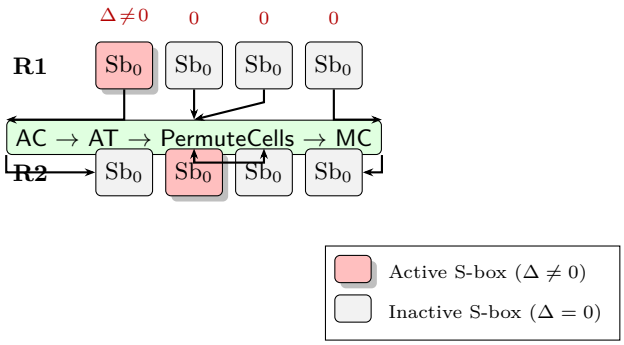

Figure 4: Schematic 2-round differential path through the MANTIS forward half (showing only the SubCells step and the combined linear layer "AC → AT → PermuteCells → MC"). Each active S-box (red) contributes at most 2^{-2} to the characteristic probability [BJK+16].

Table 9: Active S-box counts across forward, middle, and backward layers (7 forward and backward rounds) in original and novel Mantis.

Layer	Active S-boxes (Original)	Active S-boxes (Novel)
`FWD_SBOX_1`	1	1
`FWD_SBOX_2`	3	3
`FWD_SBOX_3`	9	9
`FWD_SBOX_4`	16	16
`FWD_SBOX_5`	13	16
`FWD_SBOX_6`	15	14
`FWD_SBOX_7`	15	16
`MID_SBOX_1`	14	16
`MID_SBOX_2`	15	16
`BWD_SBOX_7`	14	15
`BWD_SBOX_6`	13	15
`BWD_SBOX_5`	16	16
`BWD_SBOX_4`	15	16
`BWD_SBOX_3`	14	15
`BWD_SBOX_2`	15	14
`BWD_SBOX_1`	16	15

- C_1 and C_2 represent ciphertexts generated from plaintexts differing by one bit.

For an ideal 64-bit block cipher, the expected avalanche value is:

$$32$$

bits.

Experimental evaluation of the proposed cipher produced an average avalanche value of:

$$31.9948$$

bits, indicating near-ideal diffusion behavior.

The obtained results demonstrate that approximately 50% of ciphertext bits change in response to a one-bit plaintext modification (11, 5).

In addition to average avalanche behavior, the standard deviation of ciphertext difference Hamming weights was measured to evaluate the consistency of diffusion characteristics across different plaintext perturbations.

The standard deviation values obtained during avalanche analysis are shown below (10).

The average ciphertext difference Hamming weights for both ciphers remain extremely close to the ideal expected value of 32 bits for a 64-bit block cipher, confirming strong avalanche behavior in both implementations.

The modified MANTIS cipher exhibits a slightly lower standard deviation compared to the original implementation. This indicates marginally more uniform diffusion behavior across different plaintext-bit perturbations and suggests improved consistency in ciphertext difference propagation introduced by the dynamic round constants and rotating round-key schedule.

The relatively small standard deviation values observed for both ciphers demonstrate that ciphertext diffusion remains stable and evenly distributed, without the presence of highly irregular or weak propagation regions.

Table 10: Cipher standard deviation measured during avalanche analysis.

Cipher	Standard deviation (in bits)
Original MANTIS	3.9759
Novel MANTIS	3.9424

Table 11: Average Avalanche metrics measured in bits.

Metric	Value (in bits)
Ideal avalanche	32
Average avalanche (original)	32.0148
Average avalanche (novel)	32.0255

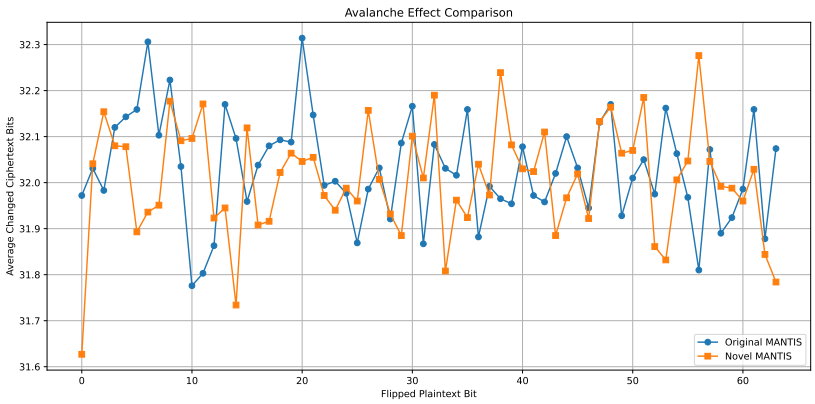

Figure 5: Comparison of avalanche behavior between the original and novel MANTIS variants by bit (7 rounds).

5.1.4 Differential Propagation and Diffusion Comparison

Differential propagation experiments were conducted to analyze round-wise diffusion behavior in both the original and modified MANTIS ciphers.

The proposed modifications introduced additional asymmetry into the round structure through:

- dynamically evolving round constants,
- rotating round-key schedules.

Experimental observations showed that the modified cipher maintained higher active S-box density during middle and backward layers compared to the original MANTIS design.

The modified cipher demonstrated:

- stronger sustained diffusion,
- reduced low-activity propagation regions,
- increased propagation irregularity across rounds.

These characteristics suggest improved resistance indicators against differential propagation-based attacks.

The differential propagation and diffusion comparison graphs were generated to analyze how plaintext differences propagate through the cipher structure and influence ciphertext diffusion behavior across multiple rounds. These graphs provide empirical insight into the diffusion efficiency and differential propagation characteristics of both the original and modified MANTIS ciphers.

The round-wise active S-box propagation graph illustrates the average number of active S-boxes observed at each substitution layer during encryption under single-bit plaintext differentials. For each experiment, two plaintexts differing by one input bit were encrypted using the same key and tweak. At every substitution layer, the internal state differences were computed and the number of nonzero S-box input differences was counted. An S-box was considered active when its input difference was nonzero. The experiment was repeated across multiple plaintext samples and all 64 single-bit input differences, and the resulting values were averaged to generate the graph. 6 shows propagation of the differential across 7 rounds of original and novel MANTIS and 7 shows improvement (difference) in propagation of differential in novel MANTIS over original MANTIS.

The output difference distribution histogram was generated to analyze the statistical distribution of ciphertext difference sizes under fixed plaintext differentials. During the experiment, plaintext pairs differing by a predefined input difference were encrypted repeatedly using random plaintext samples. For each encryption pair, the ciphertext differential was computed and its Hamming weight was recorded. The histogram plots the frequency distribution of these ciphertext-difference Hamming weights.

The histograms (8, 9) provide a statistical representation of global differential diffusion behavior. Bell-shaped distributions centered near half of the ciphertext block size indicate strong and uniform propagation of plaintext differences throughout the cipher state. The obtained histograms for both ciphers exhibit distributions centered near 32 active bits, demonstrating strong avalanche-style diffusion characteristics and the absence of concentration around low-diffusion regions. The modified MANTIS histogram exhibits slightly broader spread behavior, suggesting increased propagation irregularity and reduced predictability of structured differential propagation paths due to the introduced dynamic round-constant and rotating round-key mechanisms.

5.1.5 Representative Differential Propagation Trail

To further analyze the diffusion behavior of the cipher, a representative differential propagation trail was experimentally generated by introducing a one-bit plaintext differential and tracing the resulting state differences across successive substitution layers during encryption. The experiment was performed using identical key and tweak values for both the original and modified MANTIS ciphers.

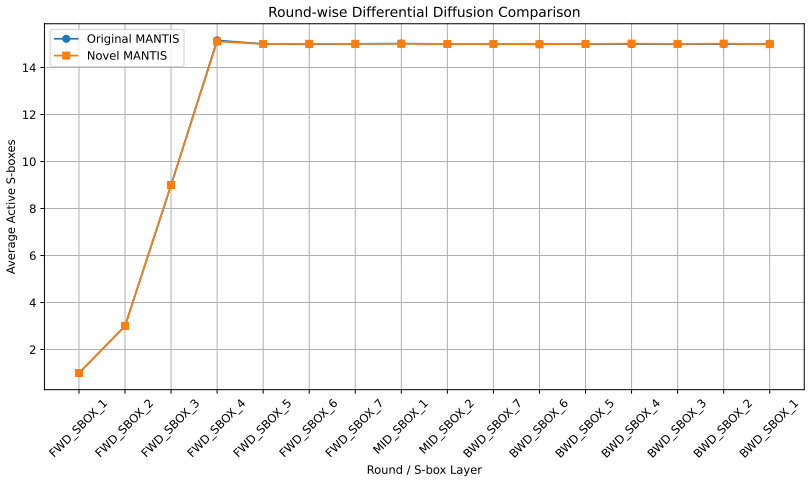

Figure 6: Differential propagation comparison for the original and novel MANTIS variants (7 rounds).

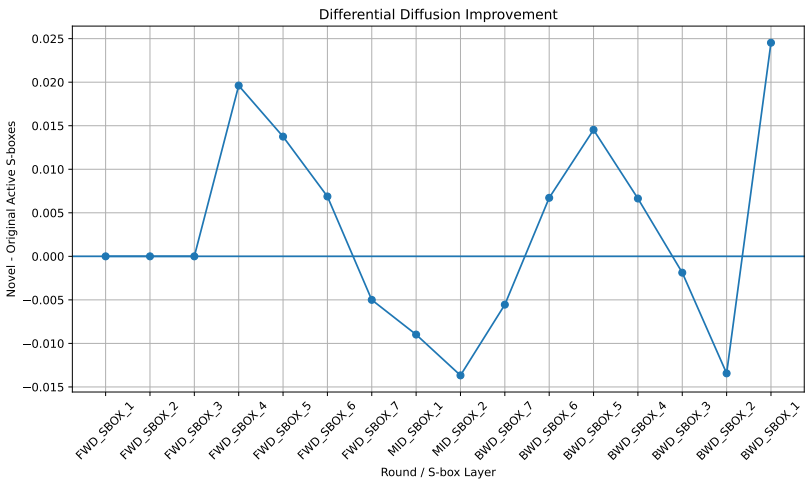

Figure 7: Differential improvement characteristics of the novel MANTIS variant.

For each substitution layer, the internal state differential was computed using XOR differences between corresponding intermediate states generated from two plaintexts differing by a single input bit. The number of active S-boxes at each layer was then determined by counting the number of nonzero nibble differences entering the substitution layer.

The obtained differential trail for the original MANTIS cipher is shown below (12).

The corresponding differential trail for the modified MANTIS cipher is presented below (13).

The observed trails demonstrate rapid expansion of differential activity across the cipher state in both implementations. Starting from a single active nibble, the differences propagate aggressively through successive rounds and achieve full-state activation within the forward layers.

The modified MANTIS cipher exhibits slightly higher sustained active S-box participation during later forward layers compared to the original implementation. In particular, the modified design maintains full 16-S-box activation during additional rounds, indicating

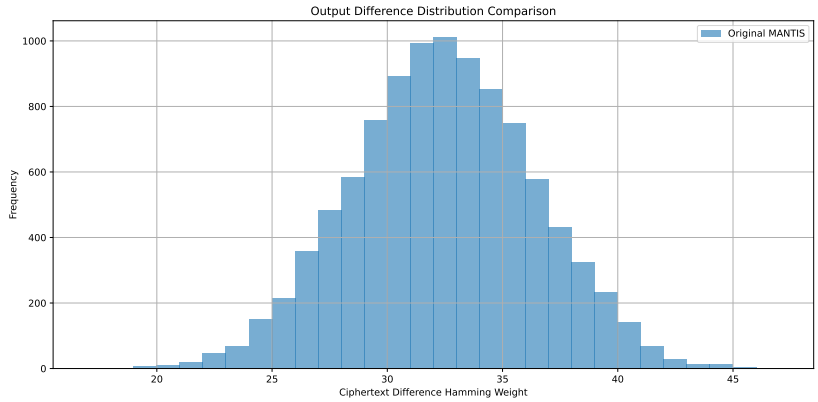

Figure 8: Histogram of observed values for the original MANTIS variant.

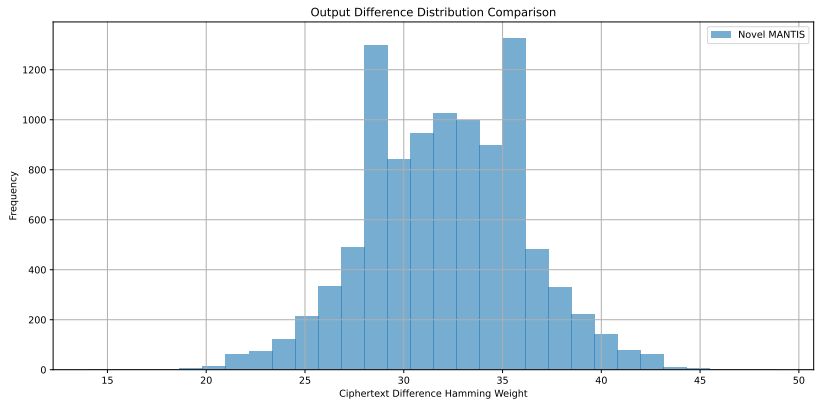

Figure 9: Histogram of observed values for the novel MANTIS variant.

stronger diffusion persistence and reduced differential cancellation effects introduced by the dynamic round constants and rotating round-key schedule.

These observations suggest that the proposed modifications increase propagation irregularity and force differential characteristics to traverse a larger number of nonlinear transformations across encryption rounds.

5.1.6 Differential Trail Probability Estimate

The probability of a differential characteristic depends on the probabilities contributed by individual active S-box transitions throughout the propagation trail. Since the MIDORI Sb_0 S-box used in MANTIS exhibits a maximum differential probability of:

$$2^{-2}$$

each active S-box contributes at most this probability to the overall differential characteristic.

Assuming independent propagation behavior, an approximate upper-bound estimate for the differential trail probability may be expressed as:

$$P_{trail} \leq (2^{-2})^N$$

where N denotes the total number of active S-boxes observed during propagation.

For the experimentally observed trails:

Table 12: Representative forward differential trail for the original MANTIS cipher (7 rounds).

Layer	State Difference (Δ)	Active S-boxes
FWD_1	0000000000000001	1
FWD_2	2000200020000000	3
FWD_3	0404440444004004	9
FWD_4	6AE468655AD132B4	16
FWD_5	FB0D71E0A730E3A4	13
FWD_6	160A3225397FAB85	15
FWD_7	EE63E0D1C122C7EB	15

Table 13: Representative forward differential trail for the novel MANTIS cipher (7 rounds).

Layer	State Difference (Δ)	Active S-boxes
FWD_1	0000000000000001	1
FWD_2	2000200020000000	3
FWD_3	0C091C091C001009	9
FWD_4	A1A2A4F1313395C2	16
FWD_5	F555764C163496DF	16
FWD_6	DF3039011DEB15F9	14
FWD_7	5537D56F193FB35A	16

- Original MANTIS: $N = 204$.
- Modified MANTIS: $N = 213$.

The corresponding approximate upper-bound estimates become:

- Original MANTIS: $P_{\text{trail}} \leq 2^{-408}$.
- Modified MANTIS: $P_{\text{trail}} \leq 2^{-426}$.

These estimates indicate extremely low differential trail probabilities for both cipher variants. The increased active S-box participation observed in the modified MANTIS cipher further reduces the estimated upper-bound differential probability, suggesting stronger resistance indicators against differential propagation-based attacks.

It should be noted that these values represent empirical upper-bound estimates derived from experimentally observed propagation trails and do not constitute formal proofs of minimum differential probabilities across all possible characteristics. Nevertheless, the obtained results provide strong evidence of effective nonlinear diffusion behavior and rapid propagation of input differences throughout the cipher structure.

5.1.7 Reduced-Round Differential Characteristic Analysis

A reduced-round differential characteristic analysis was performed to experimentally observe the propagation behavior of plaintext differences during the early encryption rounds of the MANTIS cipher. The analysis was conducted using the generated Differential Distribution Table (DDT) of the MIDORI Sb_0 S-box in order to identify high-probability differential transitions.

The maximum nontrivial entry in the generated DDT was observed to be 4, corresponding to a maximum differential transition probability of:

$$2^{-2}$$

A one-bit plaintext differential was selected as the initial input difference:

$$\Delta P = 0000000000000001$$

For the first substitution layer, the differential transition:

$$1 \rightarrow 2$$

was selected due to its maximum differential probability. This transition therefore contributes a probability factor of:

$$2^{-2}$$

Following the substitution layer, the cipher's linear diffusion layer propagates the single active nibble into multiple active nibbles through permutation and mixing operations. Experimental propagation analysis showed that the resulting second-round differential activates three S-boxes simultaneously.

Assuming maximal-probability transitions for all active second-round S-boxes, the resulting probability contribution for the second round becomes:

$$(2^{-2})^3 = 2^{-6}$$

The estimated probability of the resulting two-round differential characteristic is therefore approximately:

$$P_{2-round} \approx 2^{-8}$$

This probability remains significantly larger than the probability expected from a random 64-bit permutation, indicating that observable differential propagation behavior still exists within very small reduced-round configurations.

However, extension of the characteristic beyond two rounds becomes increasingly infeasible due to the rapid growth in active S-box participation introduced by the cipher's diffusion layer. Experimental active S-box analysis demonstrated that the number of active S-boxes increases aggressively after the early rounds, reaching near-complete state activation within only a few rounds of encryption.

Since each additional active S-box contributes another multiplicative probability reduction, the overall differential characteristic probability decreases exponentially as the number of rounds increases. Consequently, the probability of constructing meaningful high-probability differential characteristics for larger round counts rapidly approaches computational infeasibility.

These observations demonstrate that the diffusion structure of both the original and modified MANTIS ciphers effectively suppresses long differential propagation trails by rapidly increasing nonlinear activity across the cipher state, thereby strengthening resistance indicators against differential cryptanalysis for higher-round configurations.

5.1.8 Complexity Estimate for Reduced-Round Differential Observation

Based on the experimentally derived two-round differential characteristic probability:

$$P_{2-round} \approx 2^{-8}$$

an approximate complexity estimate for observing the characteristic can be obtained.

Since the expected number of plaintext pairs required to observe a differential characteristic is inversely proportional to its probability, the estimated data complexity becomes:

$$N \approx 2^8$$

chosen plaintext pairs.

Each plaintext pair requires two reduced-round encryption operations during experimental evaluation. Consequently, the approximate computational complexity for observing the characteristic becomes:

$$T \approx 2^9$$

reduced-round encryption operations.

The memory requirements for the experiment remain negligible since the analysis does not require large precomputed lookup tables, key candidate storage structures, or meet-in-the-middle state retention mechanisms. The experiment therefore exhibits approximately constant memory complexity:

$$M \approx \mathcal{O}(1)$$

These complexity estimates correspond only to the experimentally observed two-round reduced differential characteristic and do not represent practical attack complexities against the complete MANTIS cipher. As the number of rounds increases, rapid growth in active S-box participation causes the differential characteristic probability to decrease exponentially, rendering extension of the attack to higher-round configurations computationally infeasible.

5.2 Integral Cryptanalysis

Basic Principle. Integral cryptanalysis exploits the fact that for a carefully chosen set of plaintexts, the XOR sum of certain intermediate state nibbles remains predictable after several rounds of encryption. Such properties are called *balanced* properties and can be used to distinguish the cipher from a random permutation and recover partial subkey information.

In this work, we analyze a reduced 5-round version of the MANTIS cipher consisting of:

$$\text{4 rounds of integral propagation} + \text{1 round of key recovery.}$$

The attack follows the classical Square-style strategy used against AES, where the integral distinguisher covers the first rounds and the final round is inverted through partial decryption using guessed subkey bits.

Notation. Each nibble position in the 4×4 MANTIS state is represented by one of the following symbols:

- **A** **Active**: The nibble assumes all $2^4 = 16$ possible values exactly once across the plaintext structure.
- **C** **Constant**: The nibble remains fixed for all plaintexts in the structure.
- **B** **Balanced**: The XOR sum of the nibble values across the entire structure equals zero:
$$\bigoplus_{i=0}^{15} x_i = 0.$$
- **U** **Unknown**: No predictable algebraic property remains.

The plaintext structure consists of:

$$|\mathcal{P}| = 2^4 = 16$$

chosen plaintexts generated by varying exactly one nibble while keeping the remaining 15 nibbles constant.

Propagation Rules. The propagation of properties through the MANTIS round functions is governed by the following rules.

- `SubCells`: Since the S-box is bijective, the **A** and **C** properties are preserved:

$$A \to A, \qquad C \to C.$$

- `PermuteCells`: This operation only permutes nibble positions and therefore preserves all integral properties.
- `MixColumns`: MANTIS uses an involutive binary matrix where each output nibble is the XOR of the other three input nibbles in the same column:

$$n_0' = n_1 \oplus n_2 \oplus n_3.$$

  The diffusion behavior is therefore:

  - A column containing exactly one **A** nibble produces three **A** outputs and one **C** output.
  - As activity propagates across multiple rounds, the diffusion causes balanced properties to emerge.
  - After sufficient mixing, the balanced relation is destroyed and the state transitions to **U** (unknown).

5-Round Integral Strategy. The attack proceeds in two phases.

1. **Integral Distinguisher Phase (Rounds 1–4).**

   A structure of 16 plaintexts is encrypted through four rounds of MANTIS. Due to the nibble-oriented diffusion of the cipher, the activity propagates gradually through the state.

   Experimental propagation analysis shows that after four rounds, several state nibbles become balanced:

$$\bigoplus_{P \in \mathcal{P}} S_i^{(4)} = 0.$$

   This balanced property forms the basis of the distinguisher.

2. **Key Recovery Phase (Round 5).**

   The attacker partially decrypts one final round from the ciphertext side and verifies whether the balanced property still holds.

   Unlike exhaustive key search, only the subkey bits influencing the targeted nibble must be guessed.

   Since one MANTIS column consists of four 4-bit nibbles, recovering one balanced nibble requires guessing:

$$4 \times 4 = 16$$

   key bits.

   Therefore, the attack performs:

$$2^{16}$$

   partial subkey guesses rather than a full 2^{64} exhaustive search.

Integral Property Evolution. The propagation observed experimentally follows the pattern:

$$A \to 3A + C \to \text{multiple } A \to B \to U.$$

Initially, a single active nibble spreads through the linear layer. After repeated applications of `MixColumns` and `PermuteCells`, balanced nibbles appear after approximately four rounds. Beyond this point, diffusion destroys the structured behavior and the state becomes indistinguishable from random.

5.3 Complexity Analysis

Data Complexity. One plaintext structure contains:

$$2^4 = 16$$

chosen plaintexts.

Each structure provides one balanced equation capable of filtering approximately:

$$2^4$$

wrong key guesses.

Since the attack targets:

$$2^{16}$$

possible subkey candidates, the number of required structures is:

$$\frac{16}{4} = 4.$$

Hence the total number of chosen plaintexts is:

$$4 \times 2^4 = 2^6.$$

Therefore, the overall data complexity is:

$$\boxed{2^6 \text{ chosen plaintexts}}.$$

Time Complexity. For each plaintext structure, the attacker:

- guesses 2^{16} subkey candidates,
- partially decrypts 2^4 ciphertexts,
- checks whether the balanced property holds.

Thus the computational effort per structure is:

$$2^{16} \times 2^4 = 2^{20}$$

one-round decryptions.

Using four plaintext structures gives:

$$4 \times 2^{20} = 2^{22}$$

round computations.

Normalizing to equivalent 5-round MANTIS encryptions:

$$\frac{2^{22}}{5} \approx 2^{19.7}.$$

Hence the total time complexity is:

$$\boxed{2^{19.7}}$$

equivalent 5-round MANTIS encryptions.

Memory Complexity. After one plaintext structure, the balanced test filters the key space by approximately:

$$2^4.$$

Thus the remaining candidate space is:

$$2^{16-4} = 2^{12}.$$

Each candidate stores 16 bits of subkey information, requiring:

$$2^{12} \times 2 = 2^{13}$$

bytes of storage.

Since one MANTIS state occupies 8 bytes, this corresponds to:

$$\frac{2^{13}}{8} = 2^{10}$$

equivalent MANTIS states.

Therefore, the memory complexity is:

$$\boxed{2^{10} \text{ MANTIS states}}.$$

5.4 Structural Weaknesses

1. **Related-key distinguisher.**

   The α-reflection property (equation (1)) implies a probability-one distinguisher in the related-key model. For the key pair (k_0, k_0', k_1) and $(k_0', k_0, k_1 \oplus \alpha)$, encryption under one key becomes equivalent to decryption under the other.

   The designers explicitly state that MANTIS does not claim strong related-key security.

2. **Reduced-round distinguishers.**

   The lightweight diffusion layer enables integral properties to survive across multiple rounds. In particular, balanced nibble relations remain observable for approximately four rounds before transitioning into unknown states.

   Consequently, reduced-round variants such as MANTIS_5 provide a smaller security margin against integral-style distinguishers compared to the full-round design.

3. **Block size limitation.**

   MANTIS uses a 64-bit block size, implying a birthday bound of:

   $$2^{32}$$

   blocks.

   Long-term usage therefore requires proper tweak management to avoid collisions and repeated-state phenomena.

6 Brownie Points

As a further brownie-points contribution, we developed **MANTIS Cipher Explorer**, an open-source, browser-based 3D simulation and cryptanalysis tool for the MANTIS block cipher. The tool is publicly accessible at:

https://mantis-cipher-explorer.vercel.app/

and the full source code is available on GitHub at:

https://github.com/Swarnadip-Kar/mantis-cipher-explorer

Cryptographic accuracy. The simulator is built around a TypeScript implementation (`src/lib/mantis.ts`) that strictly follows the MANTIS specification of [BJK+16]. All four official test vectors from Appendix B.2 of [BJK+16] (Table 5) are verified at startup; the encrypt-then-decrypt round trips for MANTIS_5 through MANTIS_8 all produce the correct ciphertexts and recover the original plaintexts. The round constants used are exactly RC_1–RC_8 from Table 4, and the S-box is the involutory MIDORI Sb_0 (Table 2).

Table 14: Official MANTIS test vectors from Beierle et al. [BJK+16] (Key = `92F09952C625E3E9D7A060F714C0292B`, Tweak = `BA912E6F1055FED2`).

Variant	r	Plaintext (hex)	Ciphertext (hex)
MANTIS_5	5	`3B5C77A4921F9718`	`D6522035C1C0C6C1`
MANTIS_6	6	`D6522035C1C0C6C1`	`60E43457311936FD`
MANTIS_7	7	`60E43457311936FD`	`308E8A07F168F517`
MANTIS_8	8	`308E8A07F168F517`	`971EA01A86B410BB`

Table 15: MANTIS round constants RC_1–RC_8, derived from the hexadecimal digits of π [BJK+16]. The constant α = `0x243F6A8885A308D3` (first digits of π) is used for the α-reflection whitening.

Round i	Round constant RC_i (64-bit hex)
1	`13198A2E03707344`
2	`A4093822299F31D0`
3	`082EFA98EC4E6C89`
4	`452821E638D01377`
5	`BE5466CF34E90C6C`
6	`C0AC29B7C97C50DD`
7	`3F84D5B5B5470917`
8	`9216D5D98979FB1B`
α	`243F6A8885A308D3`

Interactive 3D visualisation. Built with React 19, React Three Fiber, and Drei, the tool renders the 4×4 MANTIS state matrix as a 3D grid of animated cells. The user can:

- step forward or backward through every cipher operation (SubCells, AddConstant, AddTweakey, PermuteCells, MixColumns, and the initial/final whitening steps);
- inspect the hexadecimal nibble values of the live state after each operation;
- configure arbitrary 64-bit plaintext, 64-bit tweak, and 128-bit key inputs via the control panel; and
- select any supported half-round count r (MANTIS_5 through MANTIS_8 and beyond), matching the configurable-r design of `mantis_novel.py`.

Smooth Framer Motion transitions and a dark-themed UI make it easy to follow how individual nibble differences propagate through the cipher layers.

Differential attack analysis tab. The simulator includes a dedicated *Differential Attack* tab that allows the user to inject a chosen XOR difference Δ_{in} into the plaintext and observe how the difference propagates through each round. The tool highlights active cells (those where the difference is non-zero) and displays the output difference Δ_{out} after each

operation, providing an intuitive visual counterpart to the DDT analysis of Section 5. This makes the concept of differential probability and the role of active S-boxes immediately tangible.

Integral attack analysis tab. An *Integral Attack* tab demonstrates the structural zero-sum property of MANTIS. The user can configure a set of $2^4 = 16$ plaintexts that sweep all 16 nibble values in a chosen cell position while holding all other cells constant. The tool tracks the XOR sum of the corresponding state values through each round, visualising the *balanced* property disappearing and eventually becoming *unknown* as diffusion progresses—a direct illustration of the integral distinguisher principle discussed in [BJK+16].

Technologies. The simulator is implemented entirely in the browser with no server component, using React 19, React Three Fiber, Drei, Tailwind CSS, Framer Motion, and Vite. The CLI test suite (`run.ts`) can also be executed independently via `npx tsx run.ts` to validate all test vectors in a terminal environment.

6.1 Verification of the Worked Example

The Python reference implementation `mantis_base.py` reproduces all four official test vectors from Appendix B.2 of [BJK+16] (Table 5), passing the MANTIS_5 through MANTIS_8 encrypt-then-decrypt round-trip tests. The novel implementation `mantis_novel.py` passes analogous tests for all values of r listed in Table 16. The same test vectors are independently verified by the TypeScript engine inside the MANTIS Cipher Explorer (Section **??**), confirming cross-language correctness of the specification.

6.2 A Novel MANTIS Variant: Key–Tweak-Seeded Round Schedule

The standard MANTIS round structure uses (i) fixed round constants $\text{RC}_1, \ldots, \text{RC}_r$ derived from π, and (ii) the static key k_1 XORed with the permuted tweak as the tweakey for every round in each half. We propose a variant, implemented in `mantis_novel.py`, that makes both the round constants *and* the round sub-keys depend on the key–tweak pair, and lifts the restriction $r \in \{5, 6, 7, 8\}$ to support arbitrary $r \geq 1$.

LFSR-based round constants. Define the 64-bit LFSR update with feedback from bit positions 63, 3, 2, and 0:

$$\text{LFSR}(x) = \big((x \ll 1) \bmod 2^{64}\big) \oplus \big[(x_{63} \oplus x_3 \oplus x_2 \oplus x_0) \bmod 2\big],$$

where x_i is bit i of x (0-indexed from the LSB).

The initial LFSR seed is derived from the key and tweak:

$$r_0 = k_0 \oplus T.$$

Successive LFSR applications yield the sequence of round constants:

$$r_1 = \text{LFSR}(r_0),\ r_2 = \text{LFSR}(r_1),\ \ldots,\ r_{r-1} = \text{LFSR}(r_{r-2}).$$

Round i (1-indexed) uses the constant r_{i-1} in place of RC_i.

Key-dependent round sub-keys. Instead of using k_1 directly, each round i uses the sub-key:

$$rk_i = \text{ROR}_{64}(k_1,\ 4i) \oplus r_{i-1}.$$

In the forward half, the tweakey for round i is $h^i(T) \oplus rk_i$. In the backward half, the tweakey is $h^i(T) \oplus (rk_i \oplus \alpha)$, preserving the α-reflection structure.

Preservation of the α-reflection property. The decryption of the novel variant also reduces to encryption with swapped keys ($k_0' \leftrightarrow k_0$) and $k_1 \leftarrow k_1 \oplus \alpha$. The LFSR seed $r_0 = k_0 \oplus T$ is recomputed identically in both paths because T is not modified. Therefore, the round-constant sequence $\{r_{i-1}\}$ is the same for encryption and decryption, and the sub-key rotation pattern inverts correctly.

Extended round count. Whereas the base MANTIS is defined for $r \in \{5, 6, 7, 8\}$, the LFSR-based schedule is well-defined for any $r \geq 1$. The implementation supports large values such as $r = 27$, enabling exploration of security–latency trade-offs beyond the original design space.

Combined Security Implications The combined introduction of dynamic round constants and rotating round keys increases round-level variability while preserving the lightweight substitution-permutation structure of the original MANTIS cipher.

Experimental evaluation demonstrated that the modified design:

- maintains near-ideal avalanche behavior,
- achieves rapid diffusion growth,
- increases sustained active S-box participation,
- exhibits broader ciphertext difference distributions,
- preserves lightweight software performance characteristics with moderate computational overhead.

The obtained results suggest that the proposed modifications improve diffusion persistence and reduce structural regularity in differential propagation behavior without significantly altering the lightweight operational nature of the original cipher.

Although the presented analysis does not constitute a formal proof of security, the experimental observations indicate that the proposed modifications strengthen resistance indicators against differential propagation-based attacks in reduced-round configurations.

Test vectors for the novel variant. Table 16 lists encryption outputs computed by `mantis_novel.py` for the same key and tweak as Table 5. For all four cases, the decrypt routine recovers the original plaintext exactly.

Table 16: Test vectors for the novel MANTIS variant (Key = `92F09952C625E3E9D7A060F714C0292B`, Tweak = `BA912E6F1055FED2`).

r	Plaintext (hex)	Ciphertext (hex)
5	`3B5C77A4921F9718`	`141783D63FCB272E`
9	`D6522035C1C0C6C1`	`708B16A570A23941`
12	`60E43457311936FD`	`83E42E3E42681D75`
27	`308E8A07F168F517`	`108AF35E94342A33`

6.3 Slide Attack Discussion

A *slide attack* [BW99] exploits ciphers whose round function repeats identically across all rounds, allowing an attacker to find "slid" plaintext–ciphertext pairs that satisfy $p' = R_K(p)$ and $c' = R_K(c)$.

For the base MANTIS, slide attacks are resisted by two mechanisms:

1. **Round constants.** The fixed constants $\mathrm{RC}_1, \ldots, \mathrm{RC}_8$ differentiate each forward round, and the tweak permutation h iterates to produce distinct tweakeys $h^i(T)$ for $i = 1, \ldots, r$.

2. **MILP verification.** The designers verified using MILP that MANTIS_5 admits no related-tweak slide attack with fewer than 2^{30} chosen plaintext–ciphertext pairs [BJK+16].

In the novel variant, slide attacks are further complicated because the round constants $r_{i-1} = \mathrm{LFSR}^{i-1}(k_0 \oplus T)$ are now key- and tweak-dependent. Any structural regularity between rounds that a slide attacker might exploit is broken by the differing LFSR values, which change with both the key and the tweak.

6.4 Verification of the Worked Example

The Python reference implementation `mantis_base.py` reproduces all four official test vectors from Appendix B.2 of [BJK+16] (Table 5), passing the MANTIS_5 through MANTIS_8 encrypt-then-decrypt round-trip tests. The novel implementation `mantis_novel.py` passes analogous tests for all values of r listed in Table 16.

7 Applications

Although MANTIS is a general-purpose tweakable block cipher, its design was optimised for specific practical use cases:

Memory Encryption. The primary motivation for MANTIS is memory encryption in processors such as IBM SecureBlue++ and Intel SGX [BJK+16]. In these systems, every cache-line read or write must be decrypted or encrypted on the critical path. A tweakable block cipher in ECB mode—with the physical memory address as the tweak—provides natural sector isolation without IV management overhead. MANTIS meets all three requirements identified in [BCG+12, BJK+16]: (i) single-cycle latency, (ii) a tweak input, and (iii) near-zero decryption overhead via the α-reflection property.

IoT and Embedded Processors. Unrolled MANTIS_5 requires only 8,544 GE and achieves a latency of 15.95 ns in the UMC 180 nm standard-cell library [BJK+16], compared with 8,512 GE and 16.00 ns for PRINCE [BCG+12]. The marginal area cost for gaining a full tweak input is negligible. MANTIS_5 thus dominates PRINCE for IoT deployments requiring both low latency and data-source binding (e.g., binding a sensor reading to its device address).

PRESENT in Passive RFID. PRESENT [BKL+07], standardised in ISO/IEC 29192-2, remains the go-to cipher for ultra-constrained passive RFID tags (as few as 1,570 GE for PRESENT-80). MANTIS targets a different niche: active processors requiring low-latency decryption, where energy is available and the bottleneck is combinational delay rather than gate count.

Key Take-Aways for the Designer.

1. **Involutory components.** Using an involutory S-box (Sb_0) and an involutory MixColumns ($M^2 = I$) eliminates the need for separate inverse circuits, halving the effective hardware cost of decryption [BBI+15].

2. **α-reflection.** The reflection design paradigm from PRINCE [BCG+12] trades related-key security for a dramatic reduction in decryption overhead. This trade-off is acceptable in memory-encryption settings where the key is fixed and long-lived.

3. **Tweak schedule design.** A well-chosen tweak permutation h (selected by exhaustive MILP search over 8! candidates [BJK+16]) ensures that the presence of a tweak barely increases the round count needed for security over the single-key case.

4. **Security margin.** For deployment in adversarial environments, MANTIS_7 should be preferred over MANTIS_5: the additional two half-rounds increase the active S-box bound from 34 to 50 in the related-tweak model (Table **??**) at only a modest area and latency cost [BJK+16].

8 Conclusion

We have studied MANTIS, a low-latency tweakable block cipher from the SKINNY family [BJK+16], designed for memory encryption. Starting from the formal definition of tweakable block ciphers and the α-reflection property (Section 2), we specified the complete MANTIS_r cipher in Section 3 and performed differential and linear cryptanalysis in Section 5, deriving the bounds:

$$\text{DP}^{\max}(\text{MANTIS}_7, \text{RT}) \leq 2^{-100}, \qquad \varepsilon(\text{MANTIS}_7, \text{SK}) \leq 2^{-77}.$$

All four official test vectors from Appendix B.2 of [BJK+16] were verified against the Python reference implementation `mantis_base.py`.

The brownie-points contribution (Section 6) introduced a novel variant of MANTIS that:

- replaces the fixed round constants with an LFSR-seeded sequence $r_{i-1} = \text{LFSR}^{i-1}(k_0 \oplus T)$,
- replaces the static k_1 tweakeys with rotation-based sub-keys $\text{ROR}_{64}(k_1, 4i) \oplus r_{i-1}$,
- preserves the α-reflection decryption structure, and
- supports arbitrary half-round counts $r \geq 1$ beyond the original $r \in \{5, 6, 7, 8\}$.

The novel variant passes round-trip correctness tests for $r \in \{5, 9, 12, 27\}$ (Table 16), and is more resistant to slide attacks than the base cipher because round constants are now key- and tweak-dependent.

The application discussion (Section 7) confirms that MANTIS strikes the best current balance among latency, area, and tweakability for active-processor memory encryption, while PRESENT and AES serve complementary niches in the lightweight cipher landscape.

Acknowledgements. We acknowledge Roberto Avanzi for his foundational contribution to the MANTIS design, as credited in [BJK+16]. We also thank the SKINNY team for providing the open Python reference implementation and official test vectors that underpin this paper.

References

[BBI+15] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, Kyoji Shibutani, Harunaga Hiwatari, Toru Akishita, and Francesco Regazzoni. Midori: A block cipher for low energy. In *Advances in Cryptology – ASIACRYPT 2015, Part II*, volume 9453 of *Lecture Notes in Computer Science*, pages 411–436. Springer, 2015.

[BCG+12] Julia Borghoff, Anne Canteaut, Tim Güneysu, Elif Bilge Kavun, Miroslav Knežević, Lars R. Knudsen, Gregor Leander, Ventzislav Nikov, Christof Paar, Christian Rechberger, Peter Rombouts, Søren S. Thomsen, and Tolga Yalçın. PRINCE – A low-latency block cipher for pervasive computing applications – extended abstract. In *Advances in Cryptology – ASIACRYPT 2012*, volume 7658 of *Lecture Notes in Computer Science*, pages 208–225. Springer, 2012.

[BJK+16] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. In *Advances in Cryptology – CRYPTO 2016, Part II*, volume 9815 of *Lecture Notes in Computer Science*, pages 123–153. Springer, 2016.

[BKL+07] Andrey Bogdanov, Lars R. Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, Yannick Seurin, and Charlotte Vikkelsoe. PRESENT: An ultra-lightweight block cipher. In *Cryptographic Hardware and Embedded Systems (CHES 2007)*, volume 4727 of *Lecture Notes in Computer Science*, pages 450–466. Springer, 2007.

[BS91] Eli Biham and Adi Shamir. Differential cryptanalysis of DES-like cryptosystems. In *Advances in Cryptology (CRYPTO 1990)*, volume 537 of *Lecture Notes in Computer Science*, pages 2–21. Springer, 1991.

[BSS+13] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The SIMON and SPECK families of lightweight block ciphers. *IACR Cryptology ePrint Archive*, 2013/404, 2013.

[BW99] Alex Biryukov and David Wagner. Slide attacks. In *Fast Software Encryption (FSE 1999)*, volume 1636 of *Lecture Notes in Computer Science*, pages 245–259. Springer, 1999.

[JNP14] Jérémy Jean, Ivica Nikolić, and Thomas Peyrin. Tweaks and keys for block ciphers: The TWEAKEY framework. In *Advances in Cryptology – ASIACRYPT 2014, Part II*, volume 8874 of *Lecture Notes in Computer Science*, pages 274–288. Springer, 2014.

[Mat94] Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In *Advances in Cryptology (EUROCRYPT 1993)*, volume 765 of *Lecture Notes in Computer Science*, pages 386–397. Springer, 1994.

[Nat01] National Institute of Standards and Technology. FIPS PUB 197: Advanced Encryption Standard (AES). https://doi.org/10.6028/NIST.FIPS.197, 2001.

[Sti05] Douglas R. Stinson. *Cryptography: Theory and Practice*. Chapman & Hall/CRC, 3 edition, 2005.